



The National Egyptian E-learning University, Cairo, Egypt

Faculty of Computers and Information Technology

Department of Information Technology

NileGuide: AI-Powered Tourism Planner and Activity Management Platform

A Project Submitted to the Department of Information Technology, Faculty of Computers and Information Technology,

The National Egyptian E-learning University, Cairo, Egypt

In the Requirements of Graduation for the award of the degree of bachelor's in computer and information technology

BY

[Team Member 1]

[Team Member 2]

[Team Member 3]

[Team Member 4]

[Team Member 5]

Under Supervision of

[Supervisor Name]

[Assistant Supervisor Name]

Academic Year 2025-2026

ABSTRACT

NileGuide is an AI-powered tourism planning platform designed to help tourists discover activities in Egypt, organize personal travel plans, manage wishlists, and receive contextual assistance through a retrieval-augmented AI chatbot. The system is composed of three main parts: an Angular standalone frontend, an ASP.NET Core Web API backend, and a separate FastAPI RAG service. The frontend delivers the tourist journey through home, authentication, activity discovery, activity details, wishlist, schedule, map, profile, contact and chatbot screens. It also delivers the admin journey through dashboard, users management, activities management, reports and AI file management screens.

The backend manages authentication, authorization, users, profiles, cities, categories, activities, activity images, booking links, reviews, wishlists, plans, dashboards and reports using Entity Framework Core and SQL Server. The AI module allows authorized users to upload knowledge files, extract their content, split text into chunks, store metadata in MongoDB, store vector embeddings in Qdrant and generate answers from retrieved context. This documentation explains the project in a discussion-ready format, including system architecture, database relationships, API summary, UI walkthrough, user actions, feature behavior, testing scope and future work.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

Chapter 1 - Introduction

Chapter 2 - System Analysis and Requirements

Chapter 3 - System Design and Architecture

Chapter 4 - Frontend Implementation

Chapter 5 - Backend Implementation

Chapter 6 - AI/RAG Implementation

Chapter 7 - Detailed Feature Walkthrough

Chapter 8 - Graphical User Interface Screens

Chapter 9 - Testing, Challenges and Deployment

Chapter 10 - Conclusion and Future Work

References

Appendix A - API Summary

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
AI	Artificial Intelligence
JWT	JSON Web Token
RAG	Retrieval-Augmented Generation
DTO	Data Transfer Object
EF Core	Entity Framework Core
ORM	Object-Relational Mapper
SSR	Server-Side Rendering
SPA	Single Page Application
LLM	Large Language Model
UI	User Interface
UX	User Experience

Chapter 1 - Introduction

1.1 Project Overview

NileGuide is a tourism planner for Egypt. It helps tourists move from general exploration to a structured plan. The platform starts by showing curated activities, then allows the user to inspect each activity, save favourites in a wishlist, build a schedule, view locations on a map and ask an AI assistant for help. The same platform includes an administration area that allows authorized admins to manage users, activities, reports and AI knowledge files.

1.2 Motivation

Planning a trip usually requires searching across many sources to compare prices, cities, categories, opening hours, images, reviews and providers. NileGuide reduces this effort by centralizing tourism activities and planning features in one interface. From the admin perspective, the platform reduces manual content handling by providing dashboard pages and AI file management tools.

1.3 Objectives

- Create a responsive Angular web application for tourists and administrators.
- Provide dynamic activity discovery connected to backend APIs.
- Allow authenticated users to manage wishlist, schedule, profile and chatbot interactions.
- Allow administrators to manage users, activities, analytics and AI knowledge files.
- Implement a secure role-based flow using JWT access tokens.
- Separate the AI/RAG service from the main backend for scalability and maintainability.
- Document the actual project implementation without presenting Refresh Token or Remember Me as implemented features.

1.4 Project Scope

Table 1: Implemented Feature Scope

Module	Implemented Features	Main User Benefit
Authentication	Login, register, forgot password, reset password, JWT protected requests, role-based navigation	Secures the platform and separates tourist and admin access.
Activity Discovery	Activity listing, filters, cards, activity details, images, provider links, opening hours and reviews view	Helps tourists compare activities before planning.
Wishlist	Save favourite activities, view saved items, remove items and navigate to details	Allows users to shortlist experiences before scheduling.
Schedule	Plan activities by date/time, view scheduled items, calculate itinerary context and connect to map	Turns chosen activities into a practical trip plan.
Map	Interactive map, markers and itinerary side panel	Provides location awareness for scheduled or available activities.
Profile	Personal information, profile image, preferences, travel date settings and save actions	Personalizes the experience and future recommendations.
User Chatbot	Floating chat panel, message input, chat history and AI answers	Provides assistance inside the website without leaving the page.
Admin Dashboard	Statistics, management navigation, quick access to users, activities, reports and AI files	Gives administrators an overview of system state.
Admin Users	User list, search/filtering, status/role view and management actions	Supports user administration and monitoring.
Admin Activities	Activity table, filters, create/update/delete operations and image upload workflow	Keeps tourism content up to date.
Reports	Analytics cards, charts and report data from backend services	Helps admins understand usage and activity performance.
AI File Manager	Upload, list, update and delete knowledge files used by RAG	Allows the knowledge base to evolve without code changes.

Chapter 2 - System Analysis and Requirements

2.1 Target Users

Table 2: Target Users

User Type	Description	Main Needs
Tourist / Normal User	A person looking for tourism experiences in Egypt.	Discover activities, save favourites, plan schedule, view map and ask AI assistant.
Administrator	A platform manager responsible for system content and monitoring.	Manage users, activities, reports and AI knowledge files.
Visitor	A public user who has not logged in yet.	View landing/auth/contact pages and create an account.

2.2 Functional Requirements

- The system shall allow users to register and login using email and password.
- The system shall display a list of activities from the backend with filtering and searching.
- The system shall show a dynamic details page for each activity using /activities/:id.
- The system shall allow authenticated users to add and remove activities from wishlist.
- The system shall allow authenticated users to create and manage schedule/plan items.
- The system shall display activity locations on an interactive map.
- The system shall allow users to update profile data and profile picture.
- The system shall provide an AI chatbot for user assistance.
- The system shall provide admin pages for users, activities, reports and AI file management.
- The system shall allow admins to upload/update/delete knowledge files for the RAG service.

2.3 Non-functional Requirements

Table 3: Non-functional Requirements

Requirement	Implementation Direction
Security	JWT bearer token, protected backend endpoints and admin role checks.
Usability	Dark UI, clear navigation, cards, forms and responsive layout.
Maintainability	Angular services/components, ASP.NET controllers/services/DTOs and separate AI service.
Scalability	Independent RAG service, cloud database options and hosted backend.
Performance	Frontend caching/interceptors, paginated lists and vector search for AI retrieval.
Reliability	Validation, error states, reports and documented testing scenarios.

Chapter 3 - System Design and Architecture

3.1 Overall Architecture

NileGuide follows a modular architecture. The Angular frontend communicates with the ASP.NET Core Web API for business data and protected operations. The Web API communicates with SQL Server through Entity Framework Core. Image files are uploaded to Cloudinary and stored as URLs in the database. AI-related features communicate with a separate FastAPI RAG service that uses MongoDB for metadata and Qdrant for vector search.

NileGuide System Architecture

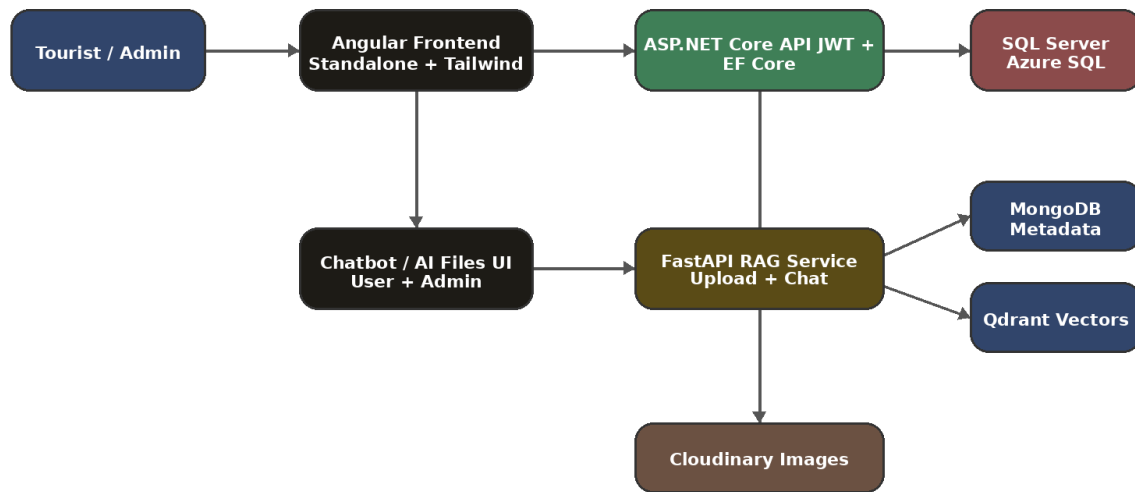


Figure 1: NileGuide system architecture

3.2 Tourist Workflow

The tourist workflow begins with discovery and authentication, then continues into activity exploration, activity details, wishlist, schedule, map, profile and chatbot support. Each screen represents a step in the planning journey.

Main Tourist Feature Workflow

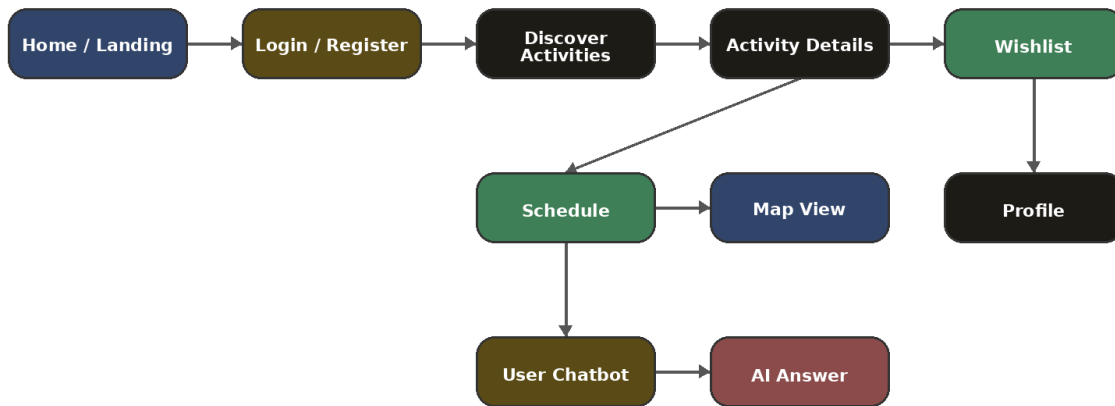


Figure 2: Main tourist feature workflow

3.3 Admin Workflow

The admin workflow starts after an admin login. The admin dashboard gives access to user management, activity management, analytics and AI knowledge file management. Admin screens are protected by role-based guards in the frontend and role authorization in the backend.

Admin Feature Workflow

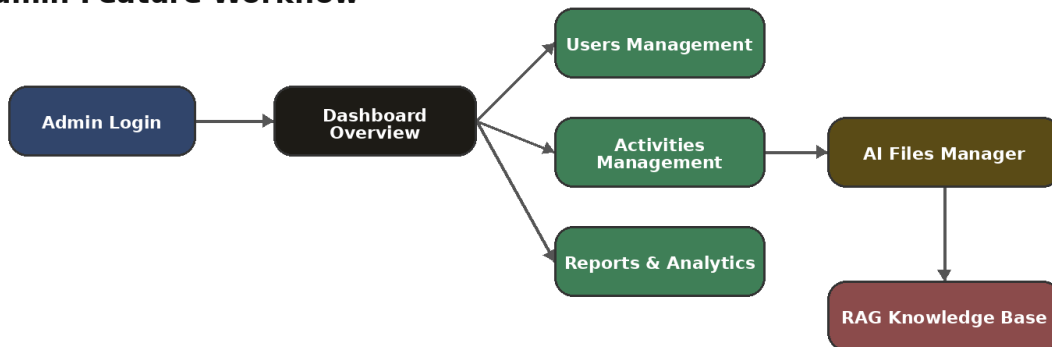


Figure 3: Admin feature workflow

3.4 Authentication Flow

The delivered authentication scope uses JWT access tokens. After login, the frontend stores the token and role, then sends the token in the Authorization header for protected API requests. Refresh Token and Remember Me are not documented as implemented features in the current project scope.

JWT Authentication Sequence

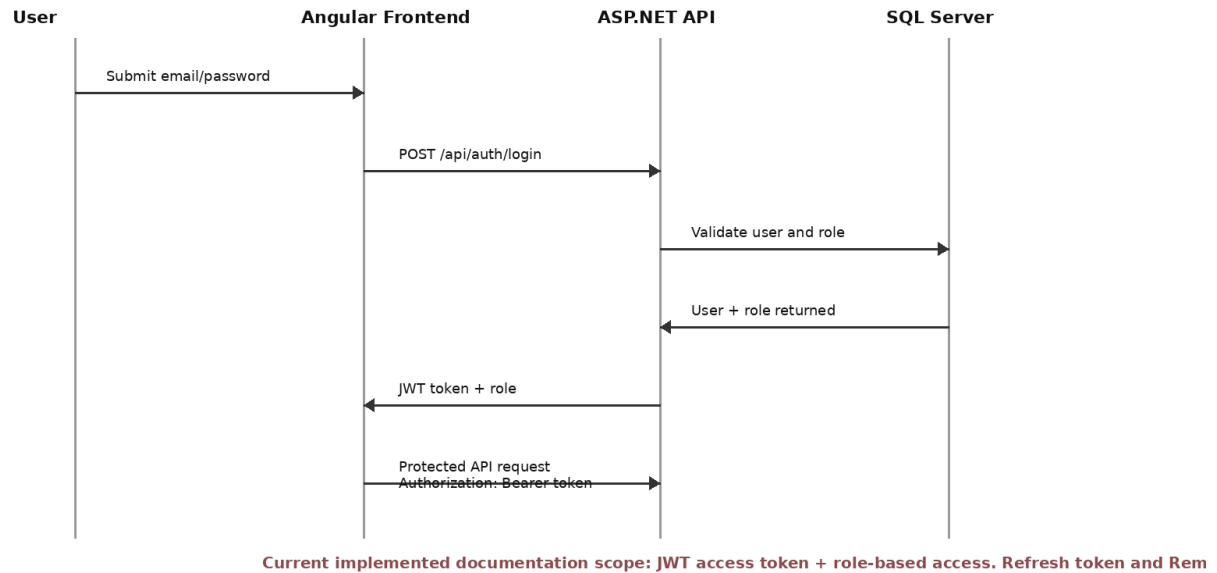
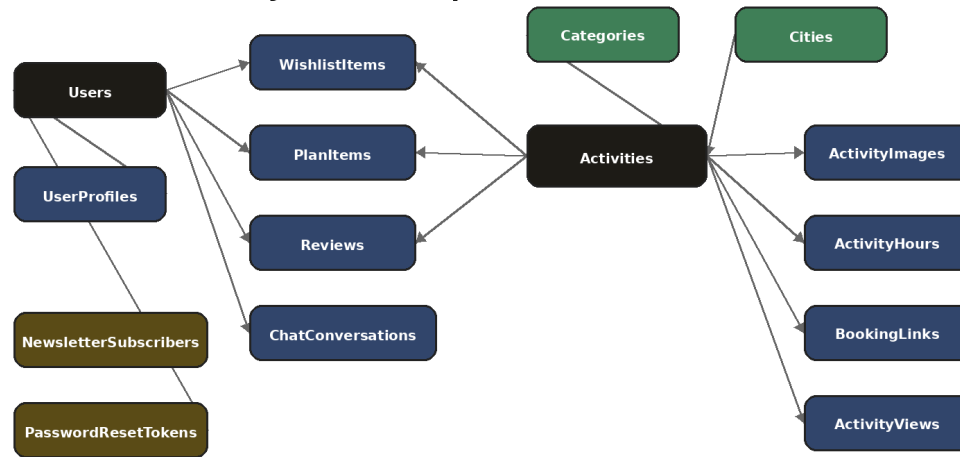


Figure 4: JWT authentication sequence

3.5 Database Design

The relational database stores core tourism and user data. Users are related to profile information, wishlist items, plan items, reviews and chat conversations. Activities are related to categories, cities, images, opening hours, booking links, views, wishlist items, plan items and reviews.

Core Database Entity Relationships



The ERD focuses on the core entities used by the delivered NileGuide modules: activity discovery, profiles, wishlist, schedule, reviews, reports, and cha

Figure 5: Core database entity relationships

Chapter 4 - Frontend Implementation

4.1 Frontend Stack

Table 4: Frontend Technologies

Technology	Usage
Angular Standalone	Main SPA structure, standalone components and routing.
Angular SSR / Hydration	Improves initial rendering behavior and supports server-side rendering configuration.
Tailwind CSS	Utility-first styling for the dark theme and responsive layouts.
Flowbite	UI components and datepicker behavior.
Font Awesome	Icons used in navbar, buttons, cards and panels.
RxJS	Asynchronous streams, API calls and search/debounce behavior.
Interceptors	Attach JWT token, handle caching or cross-cutting HTTP behavior.
Guards	Protect tourist/admin routes and redirect unauthenticated users.

4.2 Main Routes

Table 5: Main Frontend Routes

Route	Purpose
/home	Landing page and public entry.
/auth/login	User/admin login.
/auth/register	New tourist registration.
/auth/forgot-password	Request password reset.
/auth/reset-password	Complete password reset.
/activities	Activity discovery page.
/activities/:id	Dynamic activity details page.
/wishlist	Authenticated wishlist page.
/schedule	Authenticated schedule/plan page.
/map	Interactive map page.
/profile	Authenticated profile page.
/contact	Public contact page.
/dashboard	Admin dashboard entry.
/dashboard/users-management	Admin users screen.
/dashboard/activities-management	Admin activities screen.
/dashboard/reports	Admin reports screen.

4.3 Frontend Services

- AuthService handles login, register, forgot password and reset password operations.
- ActivitiesService loads activities, activity details, categories and cities.
- WishlistService manages saved activities and wishlist status.
- ProfileService loads and updates profile information and profile picture.
- Plan/Schedule service manages itinerary items and scheduled activity ids.
- Admin services handle users, admin activities, dashboard and reports.
- Chatbot/AI services communicate with the RAG chat and file management endpoints.

4.4 Frontend UI Behavior

- The navbar adapts based on authentication state and role.
- Tourist screens use cards, filters and floating chatbot controls.
- Admin screens use management tables, status cards, search fields and action buttons.
- Form screens use validation before calling backend services.
- Protected pages depend on valid JWT token and role stored by the frontend.
- Screenshots were captured from the final frontend to show the actual visual implementation.

Chapter 5 - Backend Implementation

5.1 Backend Stack

Table 6: Backend Technologies

Technology	Usage
ASP.NET Core 8 Web API	Main backend REST API.
Entity Framework Core	Database access and ORM mapping.
SQL Server / Azure SQL	Relational data storage.
JWT Authentication	Securing protected endpoints.
Role-based Authorization	Separating Tourist and Admin access.
Swagger	API testing and documentation.
Cloudinary	Profile and activity image storage.
SMTP / Email service	Forgot password and email flows.
Rate Limiting / CORS	API protection and frontend communication configuration.

5.2 Main Controllers

Table 7: Main Backend Controllers

Controller	Responsibility
AuthController	Login, registration and password recovery.
ActivitiesController	Public activity listing, details, categories/cities integration and review-related access.
AdminActivitiesController	Admin CRUD operations for activity management.
UsersController	Admin user management.
UserProfilesController	Authenticated profile data and profile picture operations.
WishlistController	User wishlist management.
PlanController	User schedule/plan management.
ReviewsController	Activity reviews.
DashboardController	Admin dashboard statistics.
ReportsController	Analytics and reporting.
ChatConversationsController	Chat conversation metadata in the .NET backend.
CategoriesController / CitiesController	Lookup data for filters and forms.

5.3 Backend Responsibilities

- Validate user credentials and issue JWT tokens.
- Validate user roles before returning admin data.
- Serve paginated and filtered activity data.
- Store activity relationships such as category, city, images, hours and booking links.
- Manage user-specific records such as wishlist, profile and schedule.
- Aggregate report data for the admin reporting page.
- Upload images and store resulting URLs in the database.

Chapter 6 - AI/RAG Implementation

6.1 Purpose of the AI Module

The AI/RAG module provides a knowledge-based chatbot. Instead of depending only on static responses, the system allows admins to upload files. The AI service extracts text from these files, converts it into searchable chunks and uses the retrieved context to generate answers.

RAG Upload and Chat Flow

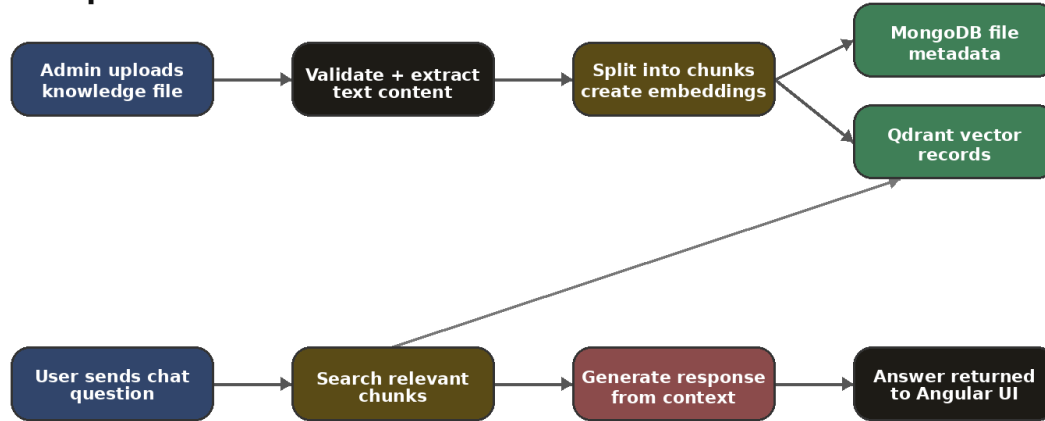


Figure 6: RAG upload and chat flow

6.2 AI/RAG Components

Table 8: AI/RAG Components

Component	Role
FastAPI	Exposes upload, list, update, delete and chat endpoints.
File extractors	Read text from PDF, DOCX, CSV and TXT files.
Chunking logic	Splits extracted text into smaller searchable parts.
Embedding model	Transforms chunks into vector representations.
MongoDB	Stores file and chunk metadata.
Qdrant	Stores vectors and enables semantic similarity search.
LLM / Generation model	Generates the final answer using retrieved chunks.
Angular Chat UI	Sends user questions and displays responses.

6.3 AI Endpoints

Table 9: AI/RAG API Endpoints

Endpoint	Method	Purpose
/	GET	Health/root endpoint.
/api/v1/data/multi-upload	POST	Upload one or more knowledge files.
/api/v1/data/files	GET	List uploaded knowledge files.
/api/v1/data/file/{file_id}	PUT	Update an existing file.
/api/v1/data/file/{file_id}	DELETE	Delete a file and related knowledge records.
/api/v1/nlp/chat	POST	Send a user question and receive an AI answer.

6.4 Supported File Types

- PDF files for documentation or guides.
- DOCX files for written knowledge sources.
- CSV files for tabular information.
- TXT files for plain text content.

Chapter 7 - Detailed Feature Walkthrough

7.1 Tourist Features

Table 10: Tourist Features by Screen

Feature	What the user does	System behavior
Authentication	Logs in or registers.	Backend validates data and returns JWT role data.
Activity Discovery	Searches and filters activities.	Frontend calls activities/cities/categories APIs and renders cards.
Activity Details	Reviews full activity information.	Frontend loads selected activity by route id.
Wishlist	Saves interesting activities.	Wishlist endpoints store user-activity relationship.
Schedule	Builds a travel plan.	Plan endpoints store scheduled items and return itinerary data.
Map	Views activity locations.	Map uses backend coordinates from activity records.
Profile	Updates personal and preference data.	Profile endpoints update the user profile and image.
Chatbot	Asks questions.	FastAPI RAG retrieves relevant knowledge and returns an answer.

7.2 Admin Features

Table 11: Admin Features by Screen

Feature	What the admin does	System behavior
Dashboard	Checks system statistics and navigates to modules.	Backend returns overview data for admin.
Users Management	Searches and manages users.	Admin endpoint returns user list and protected actions.
Activities Management	Creates, updates, deletes and filters activities.	Admin activity endpoints apply CRUD and image upload.
Reports	Reviews analytics.	Reports service aggregates data from backend.
AI File Manager	Uploads/updates/deletes AI files.	FastAPI processes files and updates MongoDB/Qdrant knowledge base.

7.3 End-to-End User Scenario

1. A visitor opens the home page and understands the NileGuide purpose.
2. The visitor creates an account through the registration page or signs in through login.
3. The tourist opens the activities page and filters by city/category/search needs.
4. The tourist opens an activity details page to inspect images, description, hours and provider links.
5. The tourist adds interesting activities to wishlist.
6. The tourist schedules selected activities and reviews them in the schedule page.
7. The tourist opens the map page to understand where planned activities are located.
8. The tourist updates profile preferences so future activity filtering/recommendations are more relevant.
9. The tourist asks the chatbot a question and receives an answer based on the uploaded AI knowledge files.

7.4 End-to-End Admin Scenario

10. Admin logs in using an admin account.
11. Admin checks dashboard statistics and navigates to management screens.
12. Admin reviews users and checks user status from Users Management.
13. Admin adds or updates an activity, including description, price, category, city, status and image data.
14. Admin reviews reports to understand platform activity and content performance.
15. Admin uploads or updates AI files so the chatbot can answer using the latest knowledge.

Chapter 8 - Graphical User Interface Screens

This chapter explains each captured screen in terms of purpose, available user actions, visible features and backend/API integration. The goal is to make the documentation useful during the graduation discussion, where the team can explain not only how each screen looks, but also what the user or admin actually does on it.

8.1 Home Page

Purpose: The home page is the public entry point of NileGuide. It introduces the platform and presents the value proposition of planning an Egyptian trip through curated activities.

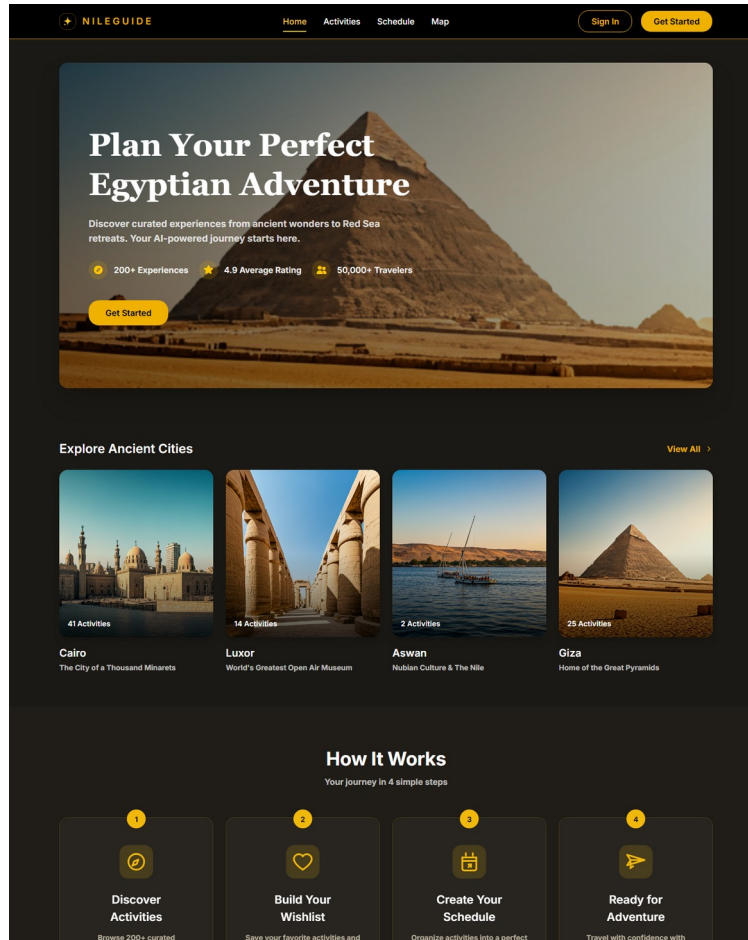


Figure 7: Home page with hero section, featured activities and navigation

What the user/admin can do:

- Read the hero message and understand the platform purpose.
- Use navigation links to move to Activities, Contact, Authentication or other public pages.
- Open the user chatbot floating button for assistance.
- Browse highlighted/featured activity content and start exploring the system.

Visible features:

- Hero section with travel-focused marketing copy.
- Featured activity cards and visual tourism content.

- Consistent navbar and footer links.
- Dark NileGuide theme with yellow/gold accent color.

Integration notes:

This screen is mainly frontend-driven and connects to the public activity discovery flow. It does not require authentication.

8.2 Login Page

Purpose: The login page authenticates existing users and administrators. It sends credentials to the backend and receives a JWT access token and role.

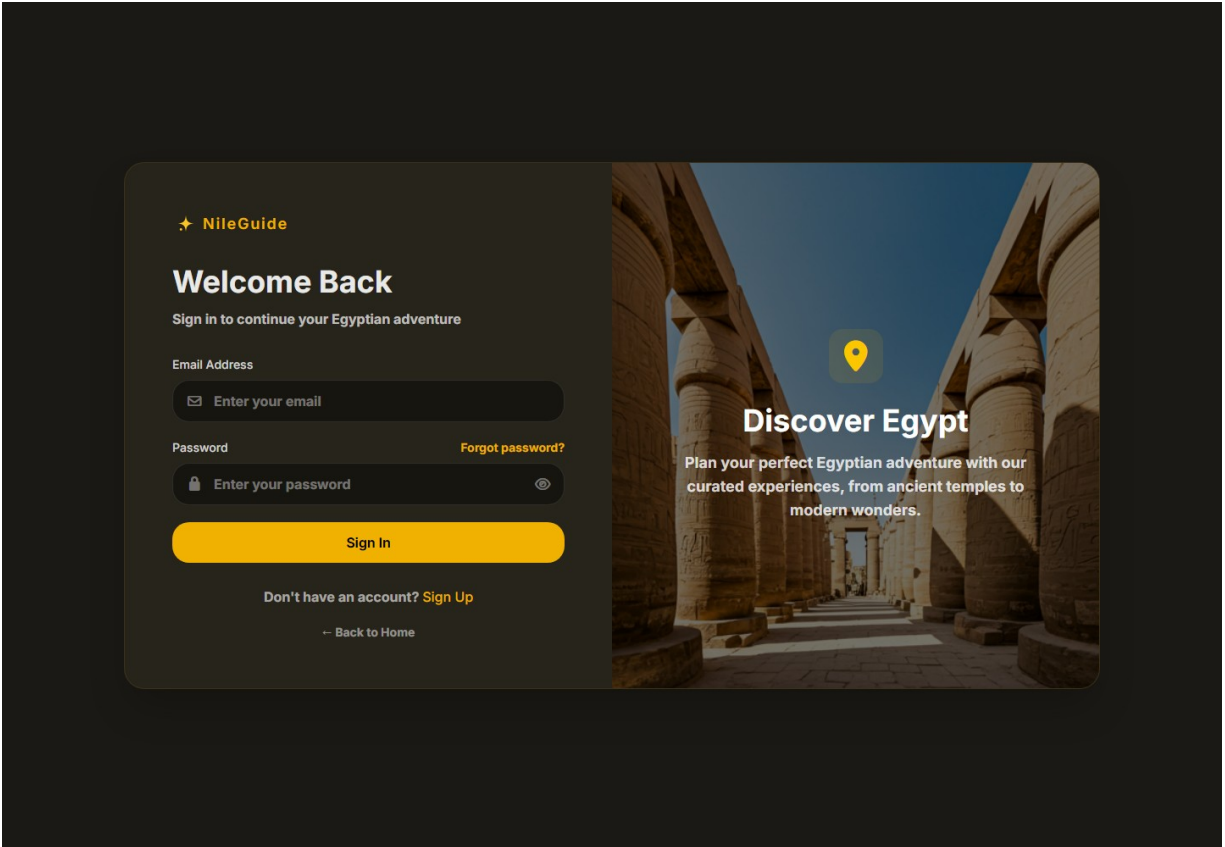


Figure 8: Login screen for tourist and admin authentication

What the user/admin can do:

- Enter email and password.
- Submit the form using the Sign In button.
- Navigate to register if the user does not have an account.
- Use the forgot password link when the password is unavailable.

Visible features:

- Reactive form validation for email/password inputs.
- JWT token storage after successful login.
- Role-based redirection to tourist or admin pages.
- Link to the forgot password workflow.

Integration notes:

Frontend AuthService calls POST /api/auth/login. Protected requests later include Authorization: Bearer <token>.

8.3 Register Page

Purpose: The register page allows new tourists to create an account and enter the basic personal data needed by the platform.

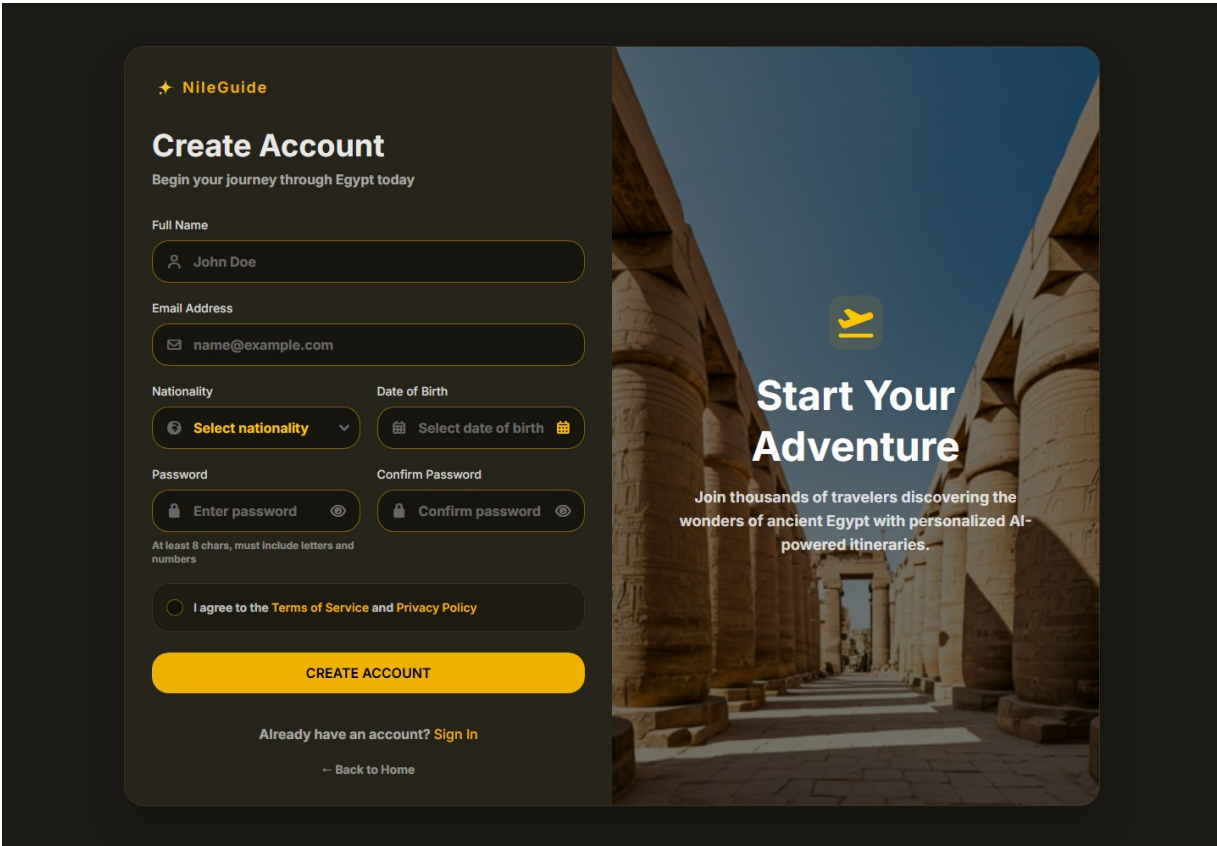
The image shows a mobile application registration screen for 'NileGuide'. The screen is split into two main sections. On the left, there is a dark-themed registration form with white and yellow text and input fields. The form includes fields for 'Full Name' (with 'John Doe' as an example), 'Email Address' (with 'name@example.com'), 'Nationality' (a dropdown menu showing 'Select nationality'), and 'Date of Birth' (a date picker showing 'Select date of birth'). Below these are 'Password' and 'Confirm Password' fields, both with 'Enter password' and 'Confirm password' as placeholders. A small note below the password fields states: 'At least 8 chars, must include letters and numbers'. At the bottom of the form is a checkbox labeled 'I agree to the Terms of Service and Privacy Policy' and a large yellow button labeled 'CREATE ACCOUNT'. Below the button, there is a link 'Already have an account? Sign In' and a small link 'Back to Home'. On the right side of the screen, there is a background image of an ancient Egyptian temple corridor with large columns. Overlaid on this image is the NileGuide logo (a yellow bird icon) and the text 'Start Your Adventure'. Below this, a smaller text reads: 'Join thousands of travelers discovering the wonders of ancient Egypt with personalized AI-powered itineraries.'

Figure 9: Registration screen for creating a new tourist account

What the user/admin can do:

- Enter full name, email, password and confirm password.
- Choose nationality and date of birth.
- Accept the terms and conditions checkbox.
- Submit the account creation form.

Visible features:

- Input validation and password confirmation.
- Date of birth input and nationality selection.
- Terms agreement before submission.
- Automatic authentication/redirect after successful registration depending on frontend logic.

Integration notes:

Frontend AuthService calls POST /api/auth/register with user registration data. The backend creates the user and returns authentication information.

8.4 Activities Page

Purpose: The activities page is the main discovery screen. It gives tourists a browsable list of tourism experiences and tools for narrowing results.

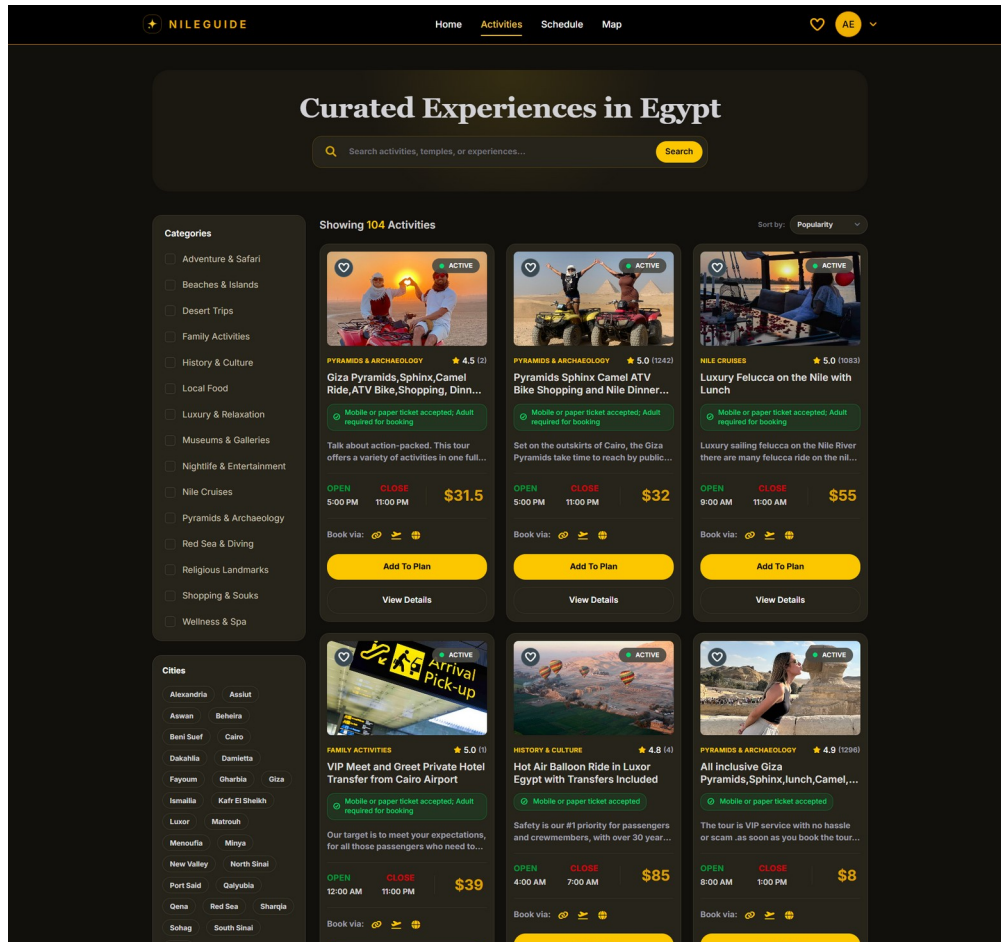


Figure 10: Activities listing page with filters, search and activity cards

What the user/admin can do:

- Search for activities using keywords.
- Filter by category, city, status or price-related options.
- Scroll through activity cards and compare images, price, city and category.
- Open activity details.
- Add or remove an activity from the wishlist.

Visible features:

- Activity cards with image, title, city/category and price information.
- Sidebar filters for category/city/status and sorting.
- Pagination or dynamic list behavior depending on loaded data.

- Wishlist icon/action connected to user state.

Integration notes:

The page calls GET /api/Activities and lookup endpoints such as GET /api/Categories and GET /api/Cities. Wishlist state is loaded using wishlist endpoints for authenticated users.

8.5 Activity Details Page

Purpose: The activity details page presents a full view of a selected activity. It is dynamic and depends on the activity id in the route /activities/:id.

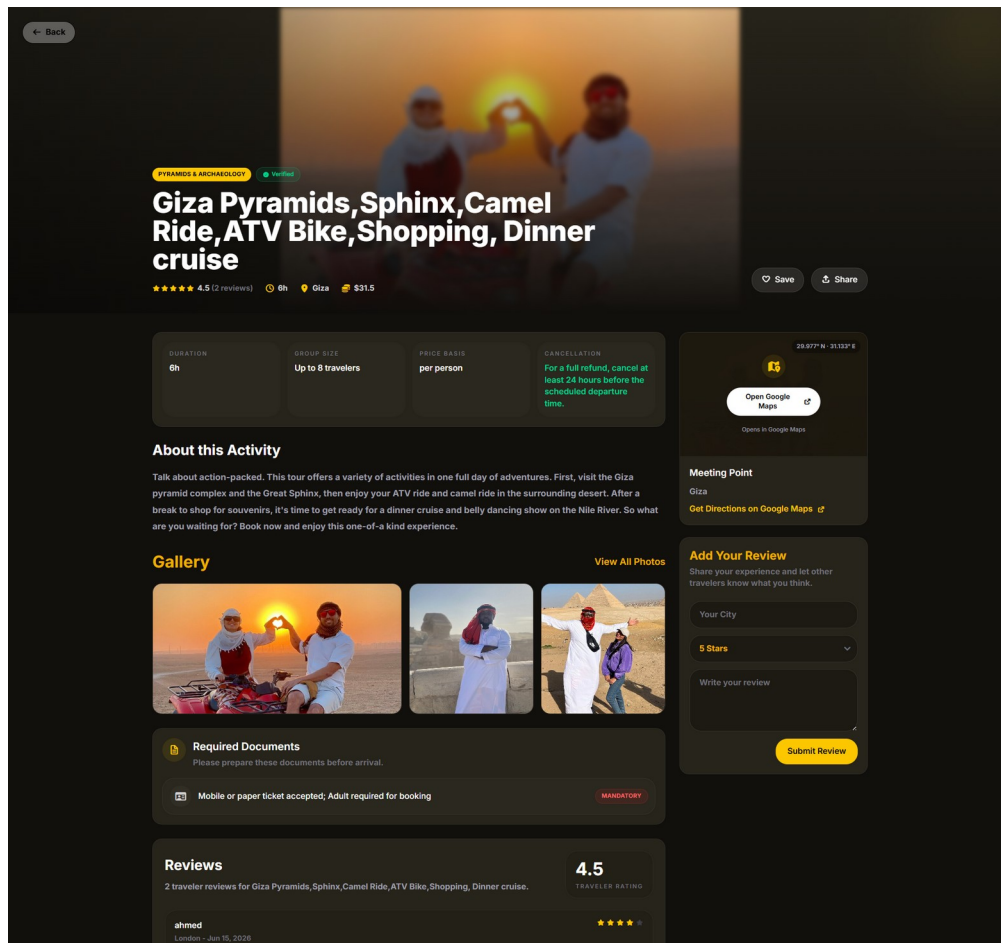


Figure 11: Dynamic activity details page

What the user/admin can do:

- Review the activity name, city, category, price and rating.
- Browse the image gallery and visual content.
- Read the description, overview, opening hours and requirements.
- Check booking providers and external provider links.
- Add the activity to wishlist or schedule.
- Read review/rating information when available.

Visible features:

- Dynamic route /activities/:id.

- Hero image and gallery thumbnails.
- Detailed information sections for overview, hours and provider data.
- Wishlist/schedule call-to-action area.
- Reviews section connected to activity-specific data.

Integration notes:

Frontend ActivitiesService calls GET /api/Activities/{id}. Related modules may call wishlist, plan and reviews endpoints for user actions.

8.6 Wishlist Page

Purpose: Wishlist stores activities the tourist is interested in but has not necessarily scheduled yet. It works as a shortlist for later planning.

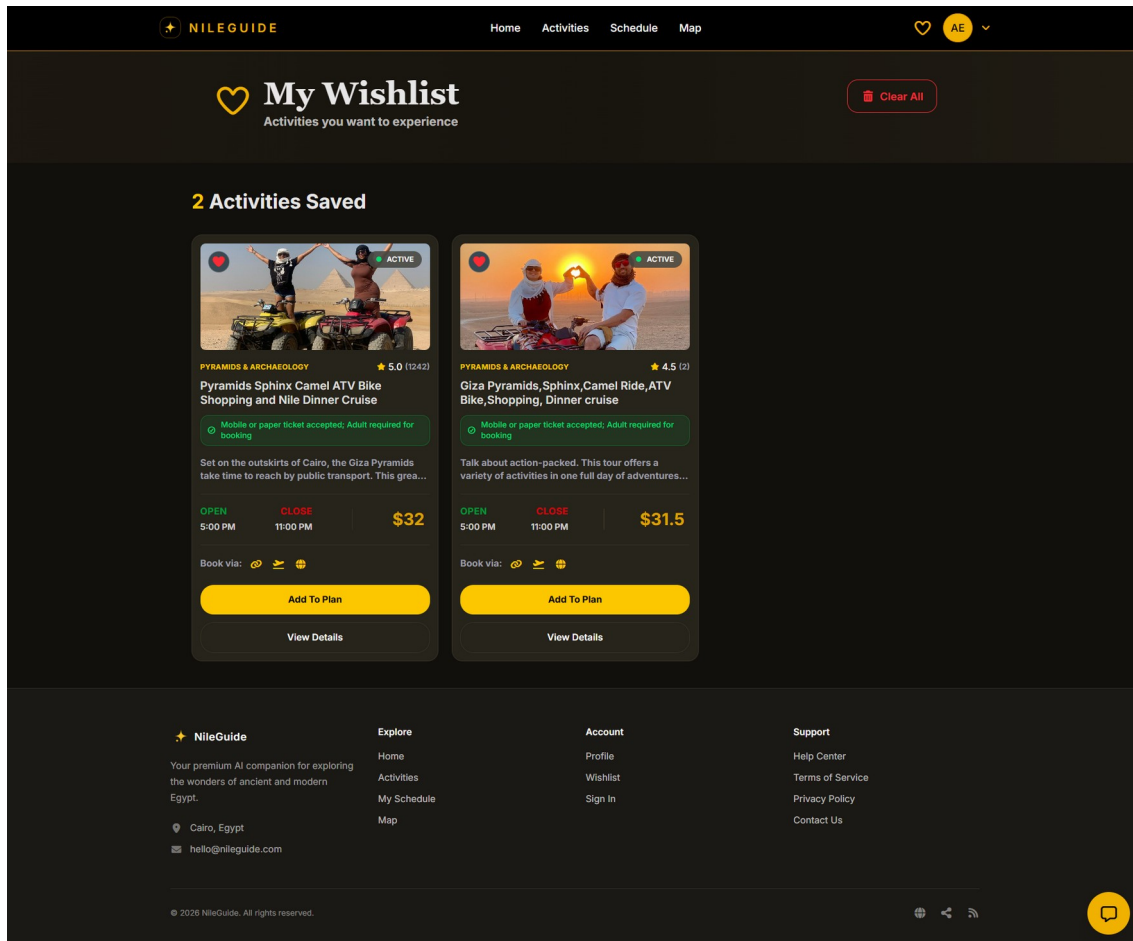


Figure 12: Wishlist page showing saved activities

What the user/admin can do:

- View all saved activities.
- Open an activity details page from a wishlist card.
- Remove an activity from wishlist.
- Use saved items as candidates for the schedule/plan.

Visible features:

- Authenticated user-specific saved activities.
- Activity cards similar to discovery cards.
- Remove/delete wishlist item action.
- Empty state handling when no activities are saved.

Integration notes:

Frontend WishlistService calls GET /api/wishlist, GET /api/wishlist/activity-ids, POST /api/wishlist/{activityId} and DELETE /api/wishlist/{activityId}.

8.7 Schedule Page

Purpose: The schedule page converts selected activities into a travel plan. It represents the user itinerary and provides structured timing for activities.

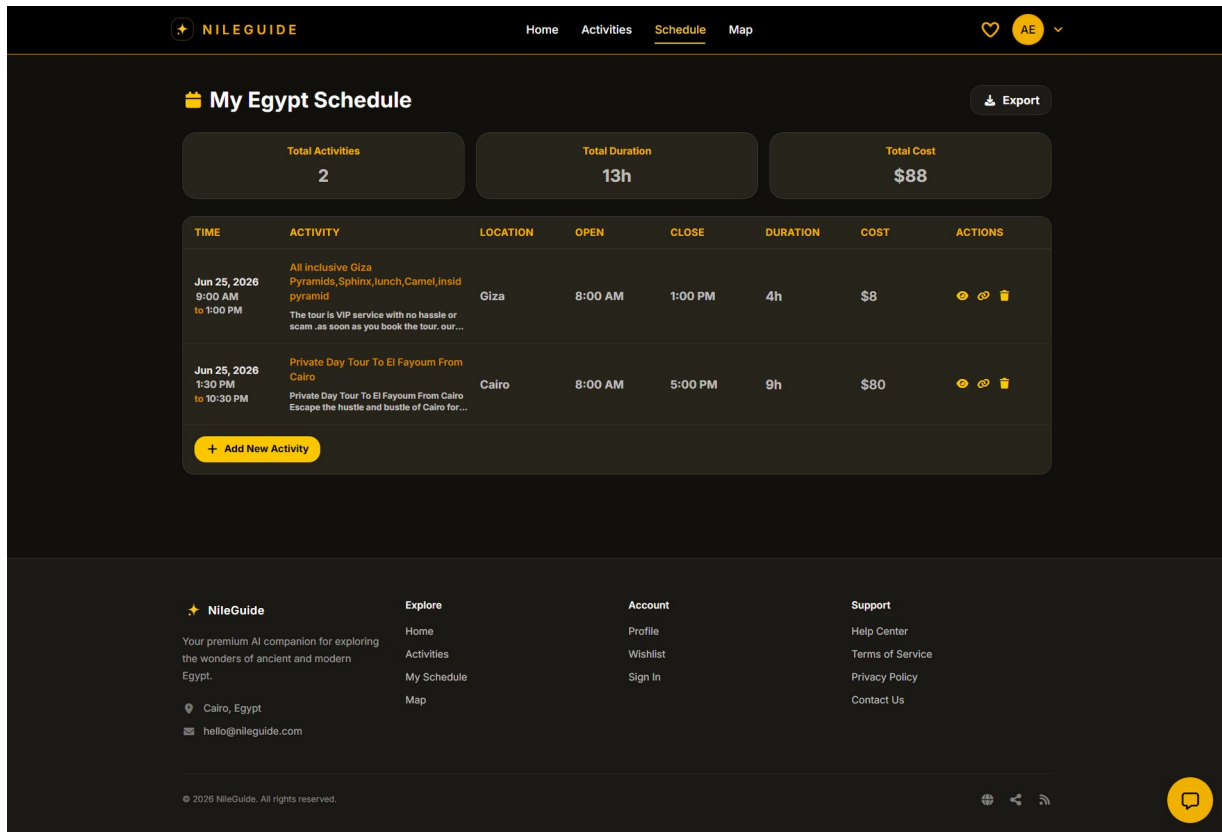


Figure 13: Schedule page for planned activities

What the user/admin can do:

- View scheduled activities with date/time context.
- Review planned activity duration, price or rating information.
- Remove activities from the schedule.
- Navigate to map view to visualize itinerary locations.

Visible features:

- Plan items grouped/displayed in a schedule table.
- Itinerary-related summary cards.
- Delete/remove schedule item actions.

- Connection between planned activities and map markers.

Integration notes:

Frontend Plan/Schedule service calls GET /api/plan, POST /api/plan/items, DELETE /api/plan/items/{planItemId} and GET /api/plan/activity-ids.

8.8 Map Page

Purpose: The map page helps the tourist understand the geographic distribution of activities and planned itinerary locations.

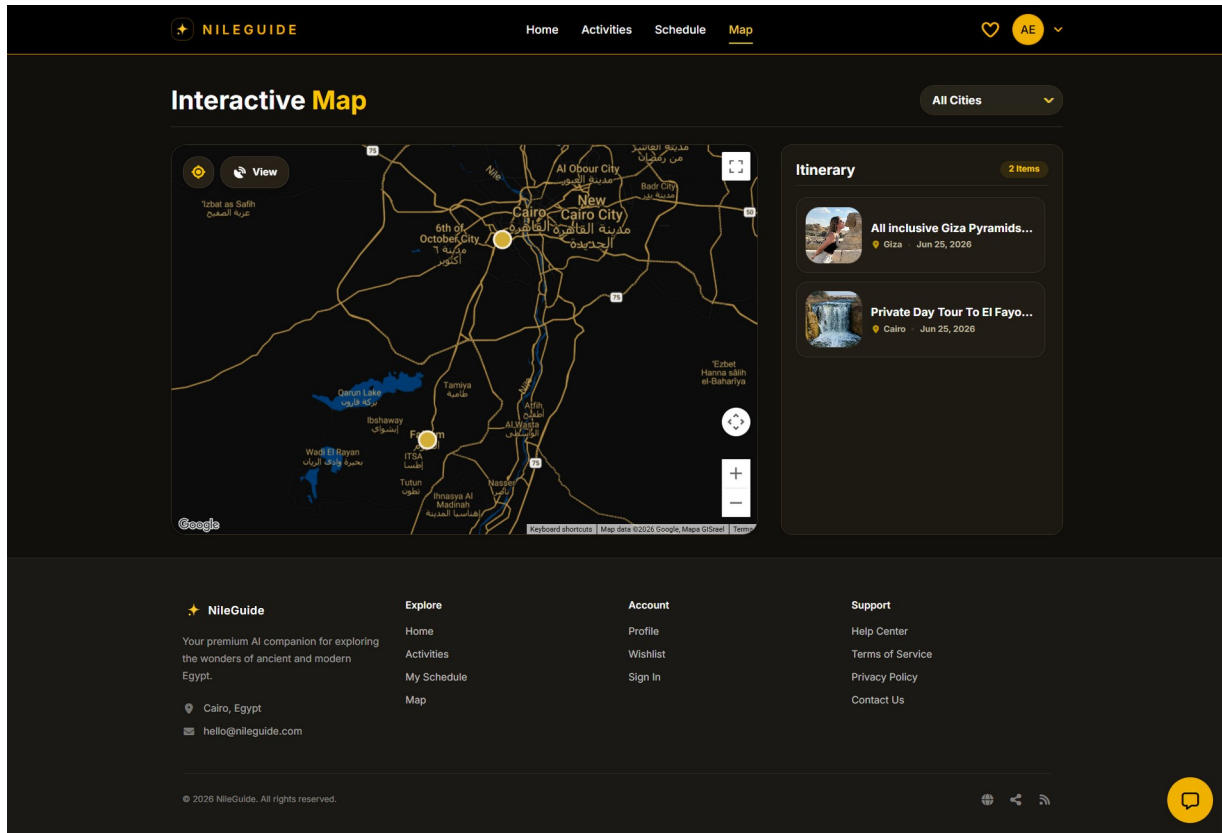


Figure 14: Interactive map page with itinerary/activity locations

What the user/admin can do:

- View activity markers on the interactive map.
- Zoom and pan to understand locations.
- Use the side itinerary panel to focus on selected activities.
- Connect activity planning with real-world geography.

Visible features:

- Map component with dark theme styling.
- Markers for activity coordinates.
- Itinerary side panel with activity cards.
- Interaction between selected items and map position.

Integration notes:

The frontend uses activity latitude/longitude returned by the backend. Map display depends on Google Maps/browser map integration and schedule/activity data.

8.9 Profile Page

Purpose: The profile page allows the tourist to manage personal details and preferences used across the platform.

The screenshot shows the 'Profile' page of the NileGuide application. The page has a dark theme with orange accents. At the top, there's a navigation bar with 'Home', 'Activities', 'Schedule', and 'Map'. A user profile icon is on the right. Below the navigation bar, a welcome message says 'Welcome to Egypt. Let's tailor your journey.' followed by a subtext: 'Tell us about yourself so our AI can craft the perfect itinerary, blending ancient wonders with modern comfort.' The main section is titled 'Basic Information' and contains fields for Name (Abdelrahman tamer emam), Email (botta@gmail.com), Date of Birth (2000-01-01), and Where are you visiting from? (Andorra). Below this is a section for travel dates with a '+ Add travel dates' button. The 'Preferred Cities' section lists 24 cities in a grid, each with a radio button. The 'What interests you most?' section shows a grid of 16 interest categories, each with a radio button. At the bottom right, there is a 'Save My Profile' button.

Figure 15: Profile page for personal information and travel preferences

What the user/admin can do:

- Edit full name, email, nationality and date of birth.
- Upload or update profile picture.
- Set travel date preference and planned visiting dates.
- Choose preferred cities and interest categories.
- Save changes and return to activity discovery.

Visible features:

- Dynamic profile loading from backend.
- Profile picture preview with fallback initials logic.
- Preference selection for cities/categories.

- Travel date controls and validation.
- Save profile action.

Integration notes:

Frontend ProfileService calls GET /api/users/me/profile, PUT /api/users/me/profile and profile picture upload endpoint. City/category lookups are loaded dynamically.

8.10 User Chatbot Panel

Purpose: The user chatbot gives tourists a convenient place to ask questions about NileGuide, travel planning or available information in the AI knowledge base.

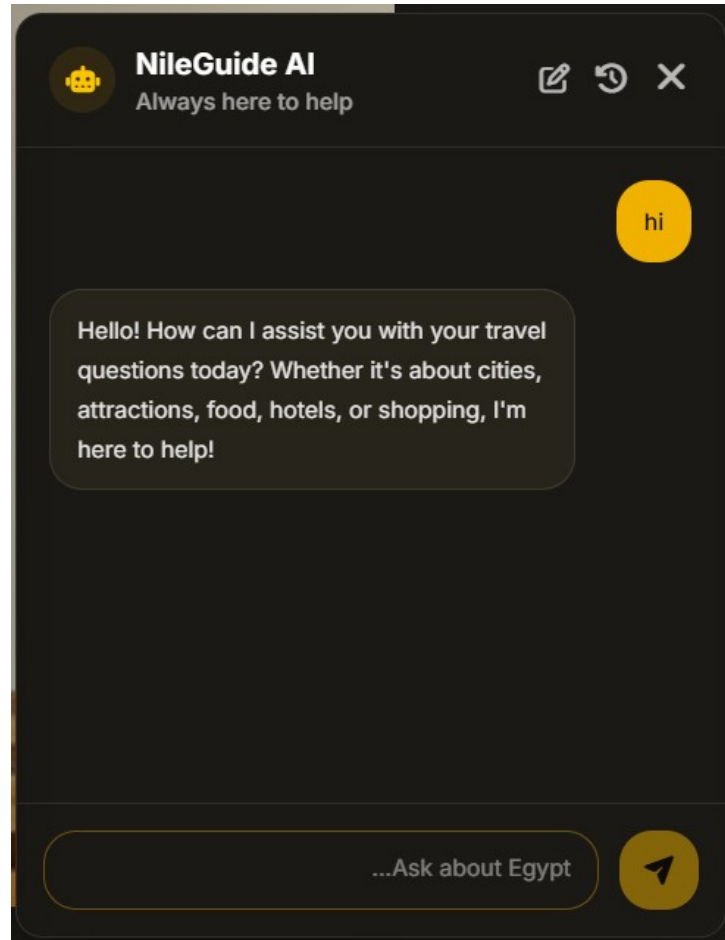


Figure 16: User chatbot panel for AI assistance

What the user/admin can do:

- Open the floating chatbot button.
- Type a travel/support question.
- Send the message and read the answer.
- Use the conversation tools/history panel when available.

Visible features:

- Floating chat widget available across user pages.
- Message input and send action.

- Chat history side view.
- Dark UI consistent with NileGuide theme.

Integration notes:

Frontend chatbot service sends the question to the FastAPI RAG chat endpoint. The AI service retrieves relevant chunks from Qdrant and returns an answer.

8.11 Chat History Panel

Purpose: The chat history panel supports continuity by displaying existing or previous chat sessions, making the chatbot feel like a persistent assistant.

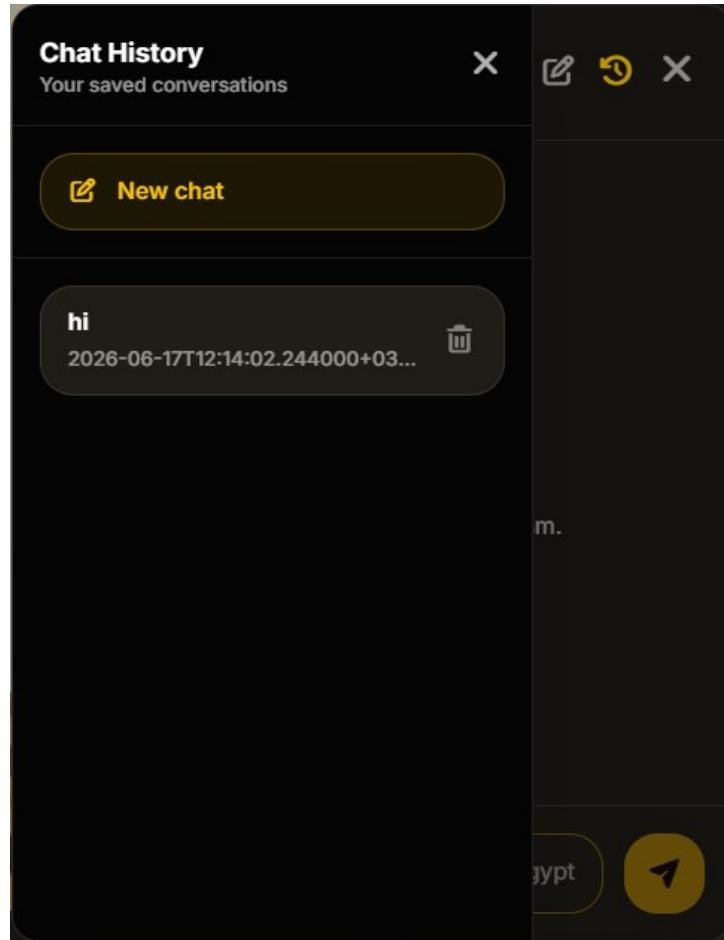


Figure 17: Chat history panel for previous conversations

What the user/admin can do:

- Open chat history.
- Start a new chat.
- Select an existing conversation if available.
- Return to the current chat panel.

Visible features:

- New chat button.
- Conversation list area.

- Close/back controls.
- Support for future conversation persistence.

Integration notes:

The AI response is handled by FastAPI RAG. Conversation identifiers or saved chat metadata can be connected through the .NET ChatConversations backend module when enabled.

8.12 Admin Users Management

Purpose: This admin screen allows administrators to monitor and manage platform users.

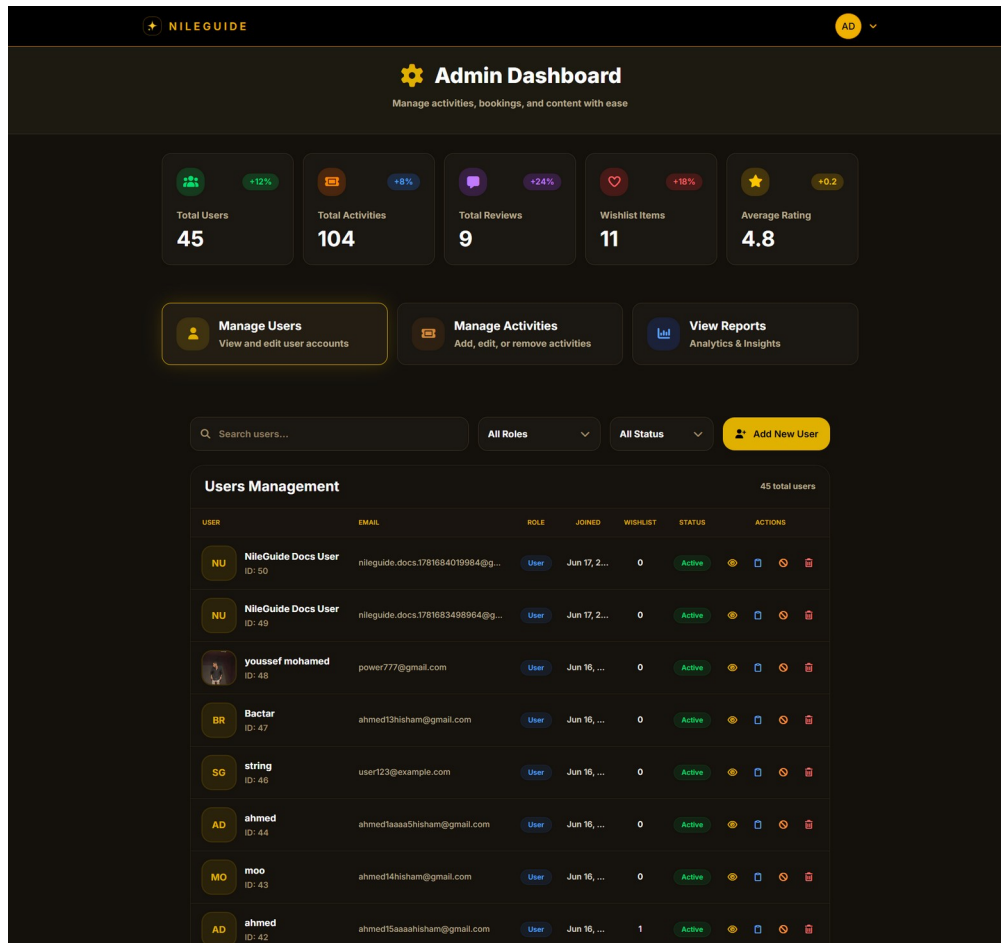


Figure 18: Admin users management screen

What the user/admin can do:

- View user statistics cards.
- Search users by name/email.
- Filter or inspect users from the table.
- Use row actions to view, edit or manage a user depending on available permissions.

Visible features:

- Dashboard-style KPI cards.
- Users table with profile/avatar information.
- Search and management action buttons.

- Admin-only access protected by role guard.

Integration notes:

Frontend Users/Admin service calls backend user management endpoints. The backend validates JWT role before returning admin data.

8.13 Admin Activities Management

Purpose: This screen is responsible for managing the tourism content that appears to users.

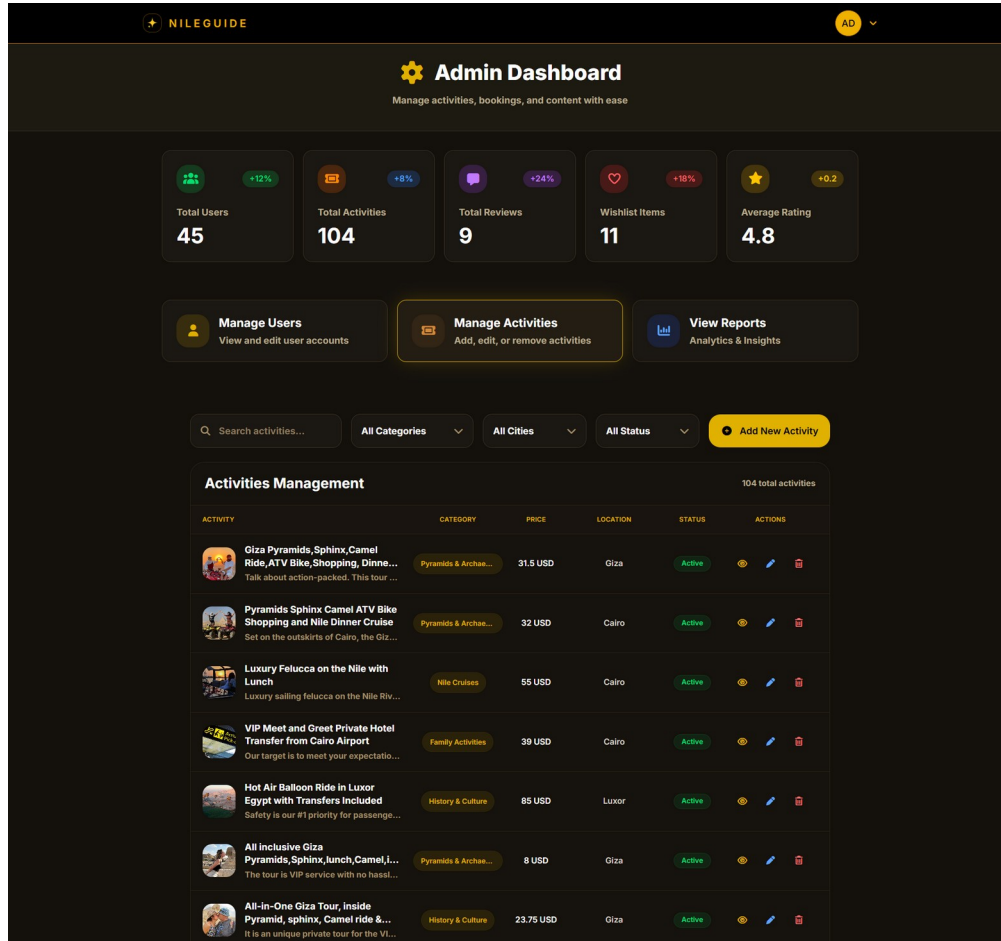


Figure 19: Admin activities management screen

What the user/admin can do:

- Search and filter activities.
- Open add activity modal.
- Update an existing activity.
- Delete an activity after confirmation.
- Manage active/inactive status and image data.

Visible features:

- Admin statistics cards.
- Activities table with city/category/status columns.
- Create/update/delete actions.

- FormData-based image upload support.
- Filters for content management.

Integration notes:

The frontend calls GET /api/admin/activities for admin listing and POST/PUT/DELETE /api/admin/activities for management operations. Images are sent through multipart/form-data.

8.14 Admin Reports

Purpose: Reports help administrators analyze platform usage and activity performance.

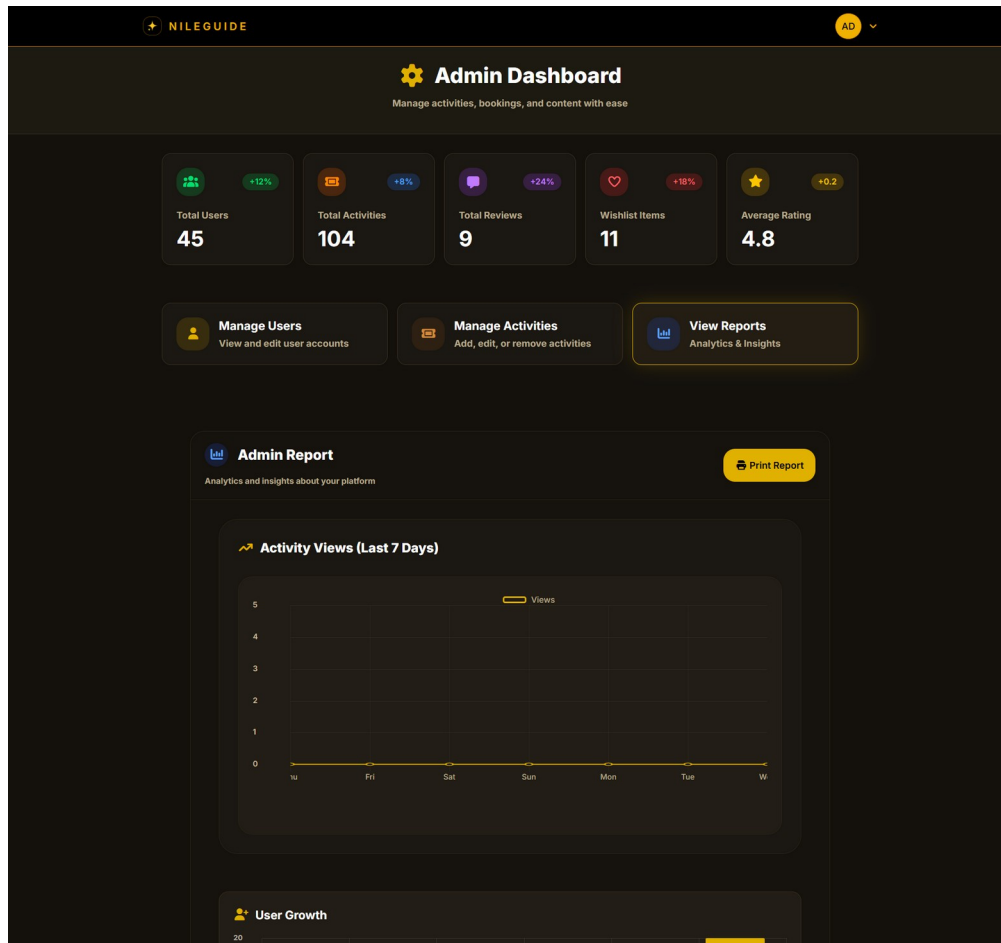


Figure 20: Admin reports and analytics screen

What the user/admin can do:

- Review dashboard report cards.
- Inspect charts such as user growth, top activities and activities by category.
- Use report data to understand which content is performing best.

Visible features:

- Charts and visual analytics.
- Report cards for high-level metrics.
- Backend-driven report data.
- Admin-only access.

Integration notes:

Frontend ReportsService calls report endpoints from the ASP.NET backend. The backend aggregates available data from entities such as users, activities and views.

8.15 Admin AI Files Manager

Purpose: This panel allows the admin to maintain the AI knowledge base by attaching source files that will be used by the RAG chatbot.

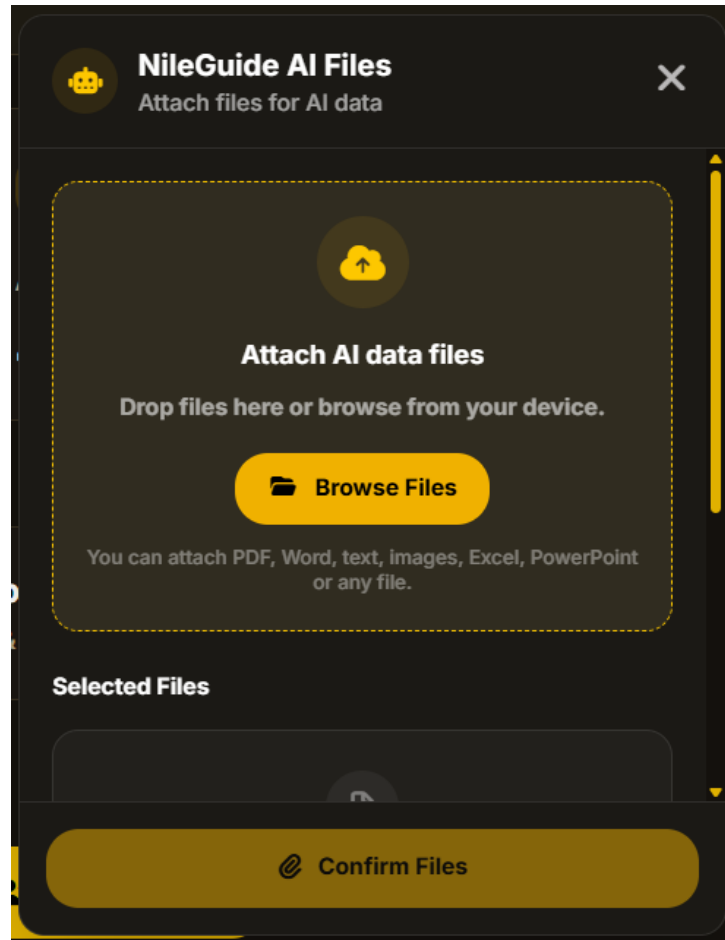


Figure 21: Admin AI file upload panel

What the user/admin can do:

- Open the AI files panel.
- Browse and select PDF, DOCX, CSV or TXT files.
- Confirm selected files for upload.
- Use file list/update/delete actions when managing existing knowledge files.

Visible features:

- Drag/drop or browse file upload area.
- Supported file type guidance.
- Confirm files action.
- Knowledge base management without changing source code.

Integration notes:

Frontend sends files to POST /api/v1/data/multi-upload in the FastAPI service. The AI service extracts text, chunks it, stores metadata in MongoDB and embeddings in Qdrant.

8.16 Contact Page

Purpose: The contact page provides a communication channel between visitors and the NileGuide team.

Figure 22: Contact page with message form and contact information

What the user/admin can do:

- Enter name, email, subject and message.
- Submit an inquiry through the contact form.
- Review contact details and support channels.

Visible features:

- Structured contact form.
- Support/contact information panel.
- Consistent public page layout.
- Footer navigation.

Integration notes:

The frontend can use EmailJS or backend-supported contact logic depending on configured implementation. The page is public and does not require authentication.

8.17 Forgot Password Page

Purpose: This page starts the password recovery workflow by allowing a user to request a reset email or reset code.

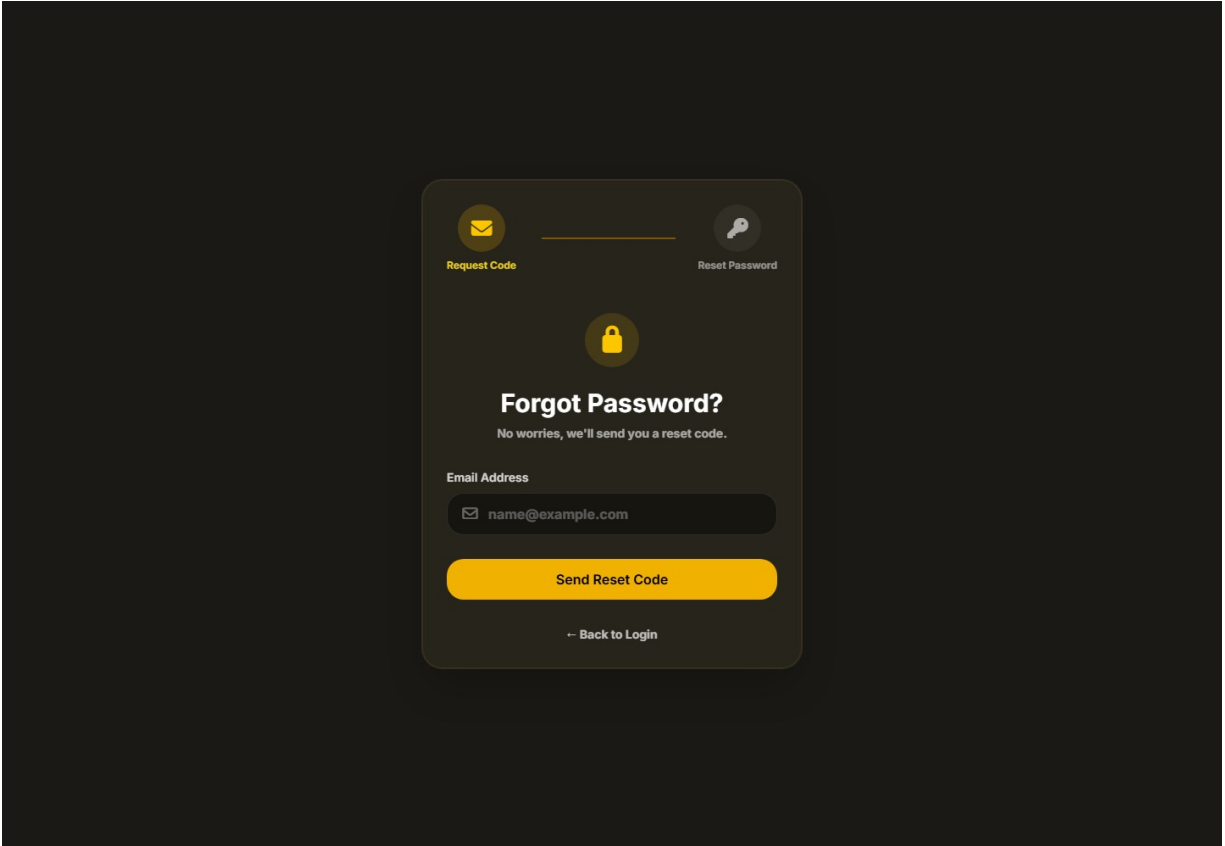


Figure 23: Forgot password request screen

What the user/admin can do:

- Enter the account email address.
- Submit the reset request.
- Navigate back to login when finished.

Visible features:

- Simple email form.
- Validation before request.
- Clear reset password call-to-action.
- Back-to-login navigation.

Integration notes:

Frontend AuthService calls a backend forgot-password endpoint. The backend prepares a reset token/code and sends it through the configured email service.

8.18 Reset Password Page

Purpose: This page completes the recovery process by letting the user set a new password using the reset token/code.

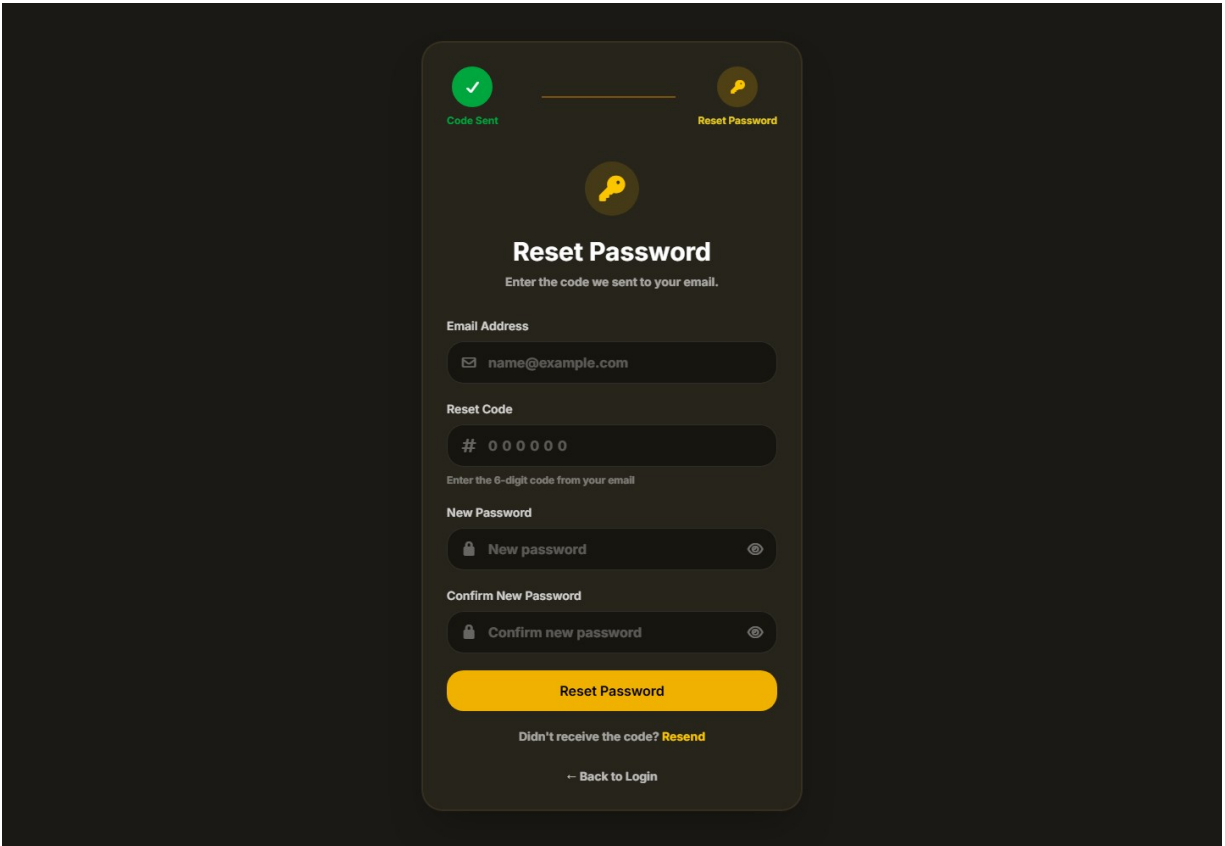


Figure 24: Reset password screen

What the user/admin can do:

- Enter email or reset code/token depending on flow.
- Enter new password and confirmation.
- Submit the form to update the password.
- Return to login after successful reset.

Visible features:

- Password and confirm-password validation.
- Reset code/token field.
- Submission button for final reset.
- Success/error handling.

Integration notes:

Frontend sends reset data to the backend reset-password endpoint. The backend validates the token/code and updates the stored password hash.

Chapter 9 - Testing, Challenges and Deployment

9.1 Testing Scope

Table 12: Testing Scope

Area	Tested Scenarios
Authentication	Login, register, forgot password, reset password and protected route behavior.
Activities	List loading, filters, details page and activity image display.
Wishlist	Add, remove, list saved items and empty states.
Schedule	Add/list/remove plan items and map integration behavior.
Profile	Load profile, update fields and upload profile picture.
Admin	Users table, activities table, report page and AI files panel.
Backend APIs	Swagger/Postman-style tests for major endpoints.
AI/RAG	Upload files, list files, delete/update files and chat response flow.
Build	Frontend build and backend publish/deployment verification.

9.2 Challenges and Solutions

Table 13: Challenges and Solutions

Challenge	Solution / Handling
Protected pages after refresh	Use JWT token from storage and route guards/interceptors. Refresh Token and Remember Me are not part of current documented scope.
SSR localStorage access	Use browser checks before localStorage access in SSR-sensitive services.
Activity image handling	Use multipart/form-data for admin create/update and store uploaded image URLs.
Dynamic details route	Use /activities/:id and load the activity from backend by id.
AI knowledge updates	Provide admin file manager so knowledge files can be updated without code changes.
Frontend/API mismatch risk	Document actual API contracts and validate request body type such as JSON vs FormData.
CORS/deployment configuration	Set correct backend base URL, CORS origins and hosted frontend URL.

9.3 Deployment View

The deployment separates the browser frontend, the ASP.NET Core backend, the SQL database and the RAG service. This separation allows the tourism system and the AI system to scale independently and makes troubleshooting easier.

Deployment View

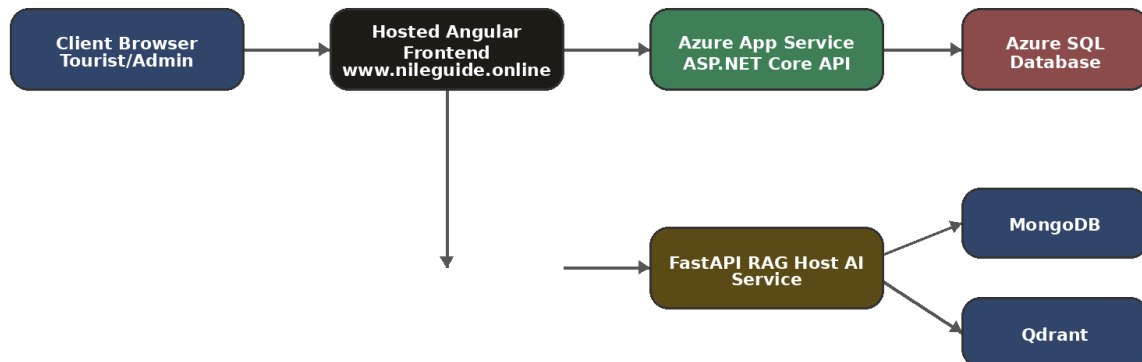


Figure 25: Deployment view

9.4 Build and Run Commands

Table 14: Common Run Commands

Part	Command / Tool	Purpose
Frontend	npm install / npm start / npm run build	Install, run and build Angular app.
Backend	dotnet restore / dotnet run / publish	Run or deploy ASP.NET Core API.
AI/RAG	uvicorn or project-specific FastAPI run command	Run RAG service.
API Testing	Swagger / Postman	Test endpoints and request/response contracts.
Screenshots	npm run docs:screenshots	Generate documentation screenshots from the frontend.

Chapter 10 - Conclusion and Future Work

10.1 Conclusion

NileGuide successfully combines tourism discovery, trip planning, admin management and AI assistance in one platform. The frontend provides a complete tourist journey and a management dashboard. The backend provides secure APIs, relational data handling and admin operations. The AI/RAG module extends the platform with file-based knowledge retrieval and contextual chat answers. The project demonstrates a practical full-stack graduation system with clear separation of responsibilities between frontend, backend and AI services.

10.2 Future Work

Table 15: Future Work

Future Feature	Description
Refresh Token / Remember Me	Can be added later for longer secure sessions, but not documented as implemented now.
Payment Integration	Allow users to book or pay for activities directly.
Advanced Recommendations	Use user preferences, history and AI to suggest personalized trips.
Notifications	Notify users about schedule reminders, updates or offers.
Multilingual Support	Support Arabic/English interface and multilingual chatbot responses.
Mobile Application	Provide a dedicated mobile app for tourists.
More Analytics	Extend reports with deeper dashboards and exportable reports.
Real Booking Integration	Connect providers directly for live availability and booking confirmation.

References

Angular Documentation - Standalone Components, Routing, HTTP Client and SSR.

ASP.NET Core Documentation - Web API, Authentication and Authorization.

Entity Framework Core Documentation - Database Context, Entities and Relationships.

FastAPI Documentation - API development and asynchronous endpoints.

Qdrant Documentation - Vector database and semantic search.

MongoDB Documentation - Document database storage.

Cloudinary Documentation - Image upload and delivery.

Tailwind CSS and Flowbite Documentation - UI styling and components.

Appendix A - API Summary

Table 16: Main API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user/admin and return JWT.
POST	/api/auth/register	Create new tourist account.
POST	/api/auth/forgot-password	Start password recovery.
POST	/api/auth/reset-password	Complete password reset.
GET	/api/Activities	Get activities list with filtering/pagination.
GET	/api/Activities/{id}	Get selected activity details.
GET	/api/Categories	Get category lookup list.
GET	/api/Cities	Get city lookup list.
GET	/api/wishlist	Get current user wishlist.
POST	/api/wishlist/{activityId}	Add activity to wishlist.
DELETE	/api/wishlist/{activityId}	Remove activity from wishlist.
GET	/api/plan	Get user schedule/plan.
POST	/api/plan/items	Add item to schedule/plan.
DELETE	/api/plan/items/{planItemId}	Remove schedule item.
GET	/api/users/me/profile	Load current user profile.
PUT	/api/users/me/profile	Update current user profile.
POST	/api/users/me/profile-picture	Upload profile picture.
GET	/api/admin/activities	Admin activity listing.
POST	/api/admin/activities	Create activity.
PUT	/api/admin/activities/{id}	Update activity.
DELETE	/api/admin/activities/{id}	Delete activity.
GET	/api/dashboard	Admin dashboard statistics.
GET	/api/reports/...	Admin report data.
POST	/api/v1/data/multi-upload	Upload AI knowledge files.
GET	/api/v1/data/files	List AI files.
PUT	/api/v1/data/file/{file_id}	Update AI file.
DELETE	/api/v1/data/file/{file_id}	Delete AI file.
POST	/api/v1/nlp/chat	Ask chatbot question.